Week # 10

Date: _____

1. Software Reuse :-

Software reuse is the process of using existing software assets, components, systems, or design knowledge to develop new software instead of building everything from the scratch.

Modern software development makes extensive use of existing software because complete re-development is often expensive and time consuming.

The objective is to reduce unnecessary redevelopment.

2. Levels of Software Reuse :-

Software can be reused at four main levels:

i) Abstraction Level :-

At this level, actual program code is not reused, Instead, previously succesful design knowledge is reused.

- Examples: design patterns, architectural patterns

ii) Object Level :-

Existing objects and classes from software libraries are reused.

- Example: using existing library classes instead of programming the same functionality again.

SP PAPER PRODUCT

Scanned with CamScanner

Date: _____

iii) **Component Level :-**

A larger reusable component or framework containing several related classes is reused.

The component may be adapted or extended to satisfy the requirements of the new system.

iv) **System Level :-**

A complete application system is reused and configured for a particular organization.

A common example is COTS software.

It is ready-made software that can be purchased and configured instead of developing the whole application from scratch.

3. Benefits of Software Reuse :-

Software reuse can :

- reduce development time
- reduce development cost
- reduce development risk
- increase productivity
- improve reliability because reused software may already have been tested and used in other systems.

SP PAPER PRODUCT

Scanned with CamScanner

Date:_____

4. Cost and Problems of Reuse :-

Reuse is not completely free. It may involve :

- searching for suitable reusable software;
- evaluating whether it satisfies the requirements;
- purchasing commercial components;
- adapting or configuring the software;
- integrating reused components with the new system.

Different reusable components may also make different assumptions about interfaces, technologies, or environments, which can make integration difficult.

- **Important Point :-**

Reuse should be considered early in software development, because the requirements or design may need to be adjusted to make effective use of available reusable software

5. Architectural Design Issues :-

Architectural design is concerned with making the major structural decisions that determine how a software system will be organized.

There is no single fixed architectural design procedure that is suitable for every system. Architectural design is mainly a process of

PAPER PRODUCT

Scanned with CamScanner

Date:

: making a series of important decisions according to the requirements and types of system being developed.

6. Major Architectural Design Decisions :-

An architect has to consider question such as :

1. Can an existing architecture be reused ?

System in the same application domain often have similar architectures.

2. How should the system be distributed ?

The architect decides whether the software runs on one computer or is distributed across different processors or machines.

3. Which architectural style or pattern should be reused ?

The organization should match the requirement of the system.

4. How should the system be decomposed ?

Large components must be divided into smaller subcomponents with clear responsibilities.

PAPER PRODUCT

Scanned with CamScanner

Date:

5. How should components communicate and be controlled ?

The architecture must define how components interact and how their execution is coordinated.

6. How will non-functional requirements be achieved ?

Architecture has strong effect on :

- performance
- security
- reliability
- maintainability
- availability

7. How should the architecture be documented and evaluted ?

The structure must be represent clearly so that it can be understood and reviewed.

7. Architectural Trade-Offs :-

Different architectural ~~designs~~ decisions may improve one quality while reducing another. For example :

- larger components may reduce communication and improve performance.
- smaller independent components may improve maintainability.

Therefore, architectural design usually requires a suitable balance between competing qualities.

PAPER PRODUCT

Scanned with CamScanner

Date:

8. Designing for Reuse :-

Sometimes a new component is developed with the intention that it may later be reused in other systems.

This is called Design for Reuse (DFR).

9. Relationship Between Reuse And Architecture :-

Reuse can occur at the architectural level because systems with similar requirements may reuse the same general architecture.

Likewise, architectural decisions should consider whether :

- an existing architecture can be reused;
- reusable components can fit into the architecture.
- an existing complete application can be adapted.

Sommerville explicitly identifies large-scale reuse as one benefit of software architecture.

SP PAPER PRODUCT

Scanned with CamScanner

Week # 11

Date: _____

1. Software Architecture :-

Software architecture is the high-level structure of a software system that describes its major components, their important properties, and the relationships or interactions among them.

Architecture focuses on the overall organization of the system, not on the detailed algorithms inside individual components.

2. Components And Connectors :-

At the architectural level, a system can be viewed as:

a) **Components**: Major software element such as:

- modules
- classes
- subsystems
- services
- databases

b) **Connectors**: Represents the interaction or relationship between components.

Examples:

- procedure or method calls
- messages
- network communication
- shared-data access
- data flow

PAPER PRODUCT

Scanned with CamScanner

Date:

3. Importance of Software Architecture :-

Software architecture is important because it:

- provides a high-level view of the whole system.
- helps stakeholders understand and discuss the system;
- records early important design decisions;
- shows how major components work together;
- strongly influences qualities such as performance, reliability, and maintainability;
- provides a foundation for detailed software design.

4. Architectural Structures :-

A software architecture can be viewed through different structures. Each structure reveals a different aspect of the same system.

i) Functional Structure :-

Shows the major functions or processing elements of the system and how they use or pass data to one another.

ii) Implementation Structure :-

Shows how

PAPER PRODUCT

Scanned with CamScanner

Date:_____

software is organized into implementation unit such as:

- packages
- classes
- objects
- procedures
- functions
- methods

iii) Concurrency Structure:-

Shows the part of the system that execute concurrently. It may contain:

- processes
- tasks
- threads
- synchronization relationships

iv) Physical Structure:-

Shows how software is related to physical hardware. It considers:

- computers or hardware nodes
- deployment
- communication interfaces
- capacity and bandwidth

v) Developmental Structure:-

Shows the software components, work products, and relationships used during software development.

PAPER PRODUCT

Scanned with CamScanner

Date:

These structures provide different architectural views depending on what information is required.

5. Architectural Styles :-

An architectural style is a general way of organizing the components and relationships of a software system.

A style defines the basic structure that guides the organization of the complex system.

i) Data-Centered Architecture :-

In a data-centered architecture, a central data store or repository is accessed by several independent components.

The components may:

- read data
- add data
- update data
- delete data

A variation is the blackboard architecture, where changes in shared data may trigger other components.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

ii) Data-Flow Architecture :-

In this, input data passes through a sequence of processing components and is gradually transformed into output. A common form is Pipe-and-Filter architecture.

- Filter performs a transformation
- Pipe transfers data from one filter to another.

Example : Source Code → Lexical Analysis → Syntax Analysis → Semantic Analysis → Code Generation.

iii) Call-and-Return Architecture :-

The system is organized as a hierarchy where one component calls another. A common form is :

Main Program → Subprogram → Lower-Level-Subprograms

It provides a clear control structure and is generally easy to understand.

iv) Object-Oriented Architecture :-

The system is organized as a collection of objects or components that :

- encapsulate data
- provide operations
- communicate with other objects.

PAPER PRODUCT

Scanned with CamScanner

Date:

v) Layered Architecture :-

The system is divided into layers, where each layer performs a related set of activities / responsibilities and generally provides services to the layer above it.

A typical arrangement is:

User Interface
↓
Application/Business Logic
↓
System Services
↓
Database/Operating System

Layers help separate responsibilities and allow changes to be localized.

6. Choosing Architectural Style :-

There is no architectural style that is best for every system. The appropriate style depends on:

- System requirements
- constraints
- application domain
- required quality characteristics.

Different styles may also be combined.

SP PAPER PRODUCT

Scanned with CamScanner

Week # 12

Date:_____

1. Architectural Patterns :-

An architectural pattern is a reusable and proven description of how a software system can be organized to solve a recurring architectural problem.

A useful architectural pattern normally explains :

- the structure of the solution
- when the pattern should be used
- its advantages
- its disadvantages

Architectural patterns allow designers to reuse successful architectural knowledge instead of inventing a completely new organization for every system.

2. Common Architectural Patterns :-

i) Model-View-Controller (MVC) :-

MVC separates an interactive system into :

- Model – manages system data and operations
- View – manages presentation of data
- Controller – manages user interaction

SP PAPER PRODUCT

Scanned with CamScanner

Date:

ii) Layered Architecture Pattern :-

In a layered pattern, related functionality is organized into separate layers. Each layer normally provides services to the layer above it.

- Advantages :-

- Supports separation of responsibilities;
- Changes can often be localized;
- Can improve portability;
- Layers may be replaced while maintaining the same interface;

- Disadvantages :-

- Clean separation between layers may be difficult.
- Passing requests through several layers may reduce performance.

iii) Repository Pattern :-

The system is organized around a central repository that stores shared data. Independent components access and modify the repository.

- Advantages :-

- easy sharing of large amounts of data.

SP PAPER PRODUCT

Scanned with CamScanner

Date:_____

- centeralized data management
- components do not need direct communication with every other component.

- **Disadvantages:-**

- all components must agree on the repository data model.
- the repository may become difficult to distribute.
- failure of the repository can affect the whole system.

iv) Client - Server Pattern :-

The system is divided into:

- Servers that provide services
- Clients that request services
- a communication network connecting them.

Examples: web server, file server, database server.

v) Pipe-and-Filter Pattern :-

Processing is divided into a segruences of independent transformations. Each filter transforms input and passes the result through a pipe to the next filter. This pattern is useful when processing naturally occurs as a segruence of transformations.

PAPER PRODUCT

Scanned with CamScanner

Date:

3. Architectural And Design Qualities :-

A good architecture must not only provide the required functionality, it must also support the required quality reports. Important qualities include :

- **Performance** : How efficiently the system responds, processes requests and use resources.
- **Reliability** : The ability of the system to operate correctly and consistently.
- **Security** : The ability to protect information and services from unauthorized access or damage.
- **Availability** : The ability of the system to remain operational and accessible when required.
- **Maintainability** : How easily the system can be corrected, changed or extended.
- **Testability** : How easily software behavior can be tested.
- **Portability** : How easily software can be moved to another platform or environment.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

- **Reusability**: How easily software elements can be reused in another system.
- **Interoperability**: The ability of the software to communicate and work with other systems.

* Architecture has a major influence on non-functional qualities such as performance, robustness, security, availability & maintainability.

4. Design Quality :-

A good software design should also be:

- traceable to requirements
- composed of well-defined components
- connected through simple and clear interface
- modular . consistent . understandable

5. Quality Trade-Offs :-

A single architecture rarely maximizes every quality.
Example:

- fewer, larger components can improve performance
- smaller, independent components can improve maintainability

Therefore, good architectural design aims for the best balance of qualities required by the system, rather than maximizing every quality.

SP PAPER PRODUCT

Scanned with CamScanner

Week #13

Date: _____

1. Quality Tactics :-

A quality tactics is an architectural or design decision used to help achieve a particular software quality attribute.

A quality attribute describes what quality is required, while a tactic describes what design decision can help achieve it.

i) Performance Tactics :-

When performance is important :

- reduce unnecessary communication between components;
- keep critical operations together;
- avoid unnecessary distribution of heavily interacting components.

Reducing overhead can improve system performance.

ii) Security Tactics :-

When security is important :

- protect critical resources
- use controlled access
- use layered protection
- place highly sensitive assets in more protected inner layers.

SP PAPER PRODUCT

Scanned with CamScanner

Date:_____

iii) Safety Tactics :-

Safety - critical functionality should be isolated into a small number of well-controlled components.

This make safety verification easier and allows special protection mechanism to be applied.

iv) Availability Tactics :-

A common tactic is redundancy. Redundant components can allow another component to continue providing a service if 1 component fails.

v) Maintainability Tactics :-

Maintainibility can be improved by using:

- small, self-contained components;
- low dependency between components;
- clear interfaces;
- separation between data producers and consumers.

2. Quality Trade-Off :-

A tactic that improves one quality can negatively affect another. Therefore, architects must select tactics according to the priority of the system's quality requirements.

PAPER PRODUCT

Scanned with CamScanner

Date: _____

3. Architecture Documentation :-

Architecture documentation records and communicates the important structure, components, relationships, and design decisions of a software system.

Different stakeholders may require different information, so architecture is often documented using several views rather than one single diagram.

- **Architectural View :-**

A view represents the architecture from a particular perspective or set of concerns.

Examples :

- logical structure
- runtime processes
- development organization
- physical deployment

4. 4+1 Architectural View Model :-

The 4+1 View Model describes architecture using four major views connected through scenarios or use cases.

PAPER PRODUCT

Scanned with CamScanner

Date:_____

- **Logical View :-**

Shows the important system abstractions, such as:

- object,
- classes,
- major logical components.

It helps relate software requirements to system elements.

- **Process View :-**

Shows the runtime organization of the software and the interaction between ~~them~~ processes.

It is useful for considering qualities such as:

- performance
- availability

- **Development View :-**

Shows how the software is divided for implementation and development teams.
It is useful to:

- programmers
- project managers
- development teams

SP PAPER PRODUCT

Scanned with CamScanner

Date: _____

- **Physical View :-** Shows the hardware environment and how software components are distributed across processors or machines.

- **+1 Scenarios/Use Cases :-** Important use cases or scenarios connect the other views and demonstrate how the architecture supports actual system behavior.

5. Ways to Document Architecture :-

It can be documented using:

a) Block Diagrams :-

Provide a simple high level picture of:

- components
- subcomponents
- data / control relationships

They are easy for different stakeholders to understand.

b) UML Diagram :-

UML may be useful for more detailed architectural documentation. It can represent:

PAPER PRODUCT

Scanned with CamScanner

Date:

- components
- interfaces
- deployment
- classes
- interaction

c) Architectural Description Language (ADL):-

An ADL is a more formal language with syntax and semantics for describing architectures including:

- components
- composition
- interfaces
- connections.

6. How Much Documentation Is Needed?

Not every project requires every possible architectural view.

The architect should develop the views that are actually useful for communication and decision-making, rather than producing documentation simply for completeness. So:

Useful Documentation > Excessive Documentation

PAPER PRODUCT

Scanned with CamScanner

Week # 14

Date: _____

1. Architectural Evalution :-

It is the process of examining a proposed architecture to determine whether it can satisfy the system's requirements and important quality attributes.

Evaluation is useful before complete implementation because architectural problems are easier and cheaper to correct at an early stage.

Architecture can be evaluated using:

- use-case scenarios;
- quality scenarios;
- change scenarios;
- quality attributes;
- component dependencies;
- architectural alternatives.

Different candidate architectures can be compared to determine which one satisfyies the requirement more effectively.

2. Architecture Trade-Off Analysis Method (ATAM) :-

It is an iterative method used to evaluate software

PAPER PRODUCT

Scanned with CamScanner

Date:

architectures by studying requirements, scenarios, quality attributes, sensitivity points, & trade-offs.

• Main ATAM Steps:-

1. Collect Scenarios: Identify important ways in which users will interact with the system.

2. Identify Requirements, Constraints and Environment: Understand stakeholder needs and technical restrictions.

3. Describe Candidate Architecture: Describe the architectural styles or patterns selected for the system.

4. Evaluate Quality Attributes: Examine qualities such as:

- performance
- reliability
- security
- maintainability
- testability
- portability

5. Identify Sensitive Points: Determine which architectural decisions strongly affect important qualities.

6. Identify Trade-Off Points and Compare Alternatives: Determine where the same architectural decision affects several quality attributes and select or improve the architecture accordingly.

SP PAPER PRODUCT

Scanned with CamScanner

Date: _____

3. Sensitivity Point :-

A sensitivity point is an architectural property or decision where a small change causes a significant change in a quality attribute.

- **Example**: The number of servers may strongly influence system performance.

4. Trade-Off Point :-

A trade-off point is an architectural decision that influences multiple quality attributes.

For example, increasing the number of servers may :

- improve performance
- improve availability
- but increase cost and complexity.

- **Easy Difference** :-

Sensitivity Point → Strongly affects one quality.

Trade-Off Point → Affects multiple qualities

SP PAPER PRODUCT

Scanned with CamScanner

Date:_____

5. Model-Driven Development :-

Model-driven development is an approach in which models are treated as major development artifacts and can be transformed into more detailed models or executable software.

The aim is to work at a higher level of abstraction instead of concentrating immediately on programming-language and platform details.

6. MDE and MDA :-

i) Model-Driven Engineering :-

A broad approach in which models may be used throughout:

- requirements
- design
- implementation
- testing

ii) Model-Driven Architecture :-

A model-focused approach mainly concerned with software design and implementation.

PAPER PRODUCT

Scanned with CamScanner

Date: _____

7. Main Models in MDA :-

MDA attempts to move from a very abstract system description towards executable software using model transformations. There are three main model types :

i) Computation Independent Model :-

Describes the important concepts of the problem/domain without implementation details. It focuses on what exists in the domain. Also called a Domain Model.

ii) Platform Independent Model (PIM) :-

Describes the system structure and behavior without depending on a particular technology or implementation platform.

Example: The model explains registration functionality without deciding whether it will use Java, .NET, etc.

iii) Platform Specific Model :-

Adds implementation details for a specific platform.

Examples:

SP PAPER PRODUCT

Scanned with CamScanner

Date:

- Java / J2EE
- .NET

The main transformation is :

CIM → PIM → PSM → Code

8. Model Transformation :-

The important idea behind MDA is that models can be transformed progressively.

A PIM may be transformed into different PSMs for different platforms.

For example :

One PIM can produce

- Java - specific model → Java Code
- .Net - specific model → C# code

This allows the same high-level model to support different implementation platforms.

PAPER PRODUCT

Scanned with CamScanner

Date:

9. Advantages of Model-Driven Development :-

Possible benefits include :

- working at a higher level of abstraction;
- less concern with low-level implementation details;
- potential reduction in errors;
- faster design/implementation;
- reusable platform-independent models;
- possibility of generating implementations for different platforms from one model.

10. Limitations :-

MDA has not become the universal approach to software development. There are several limitations including :

- modules useful for discussion may not always be ideal for implementation;
- specialized transformation tools may be required.
- tooling may be expensive to introduce or maintain.
- benefits of platform independence may mainly matter for large, long-lived systems.
- additional manual coding reduces the benefit of automatic generation;

SP PAPER PRODUCT

Scanned with CamScanner

Date:_____

- widespread Agile development has also reduced attention given to extensive up-front modeling.

PAPER PRODUCT

Scanned with CamScanner